Name: Shah Nawaz Hussain
Roll No: 244CA49

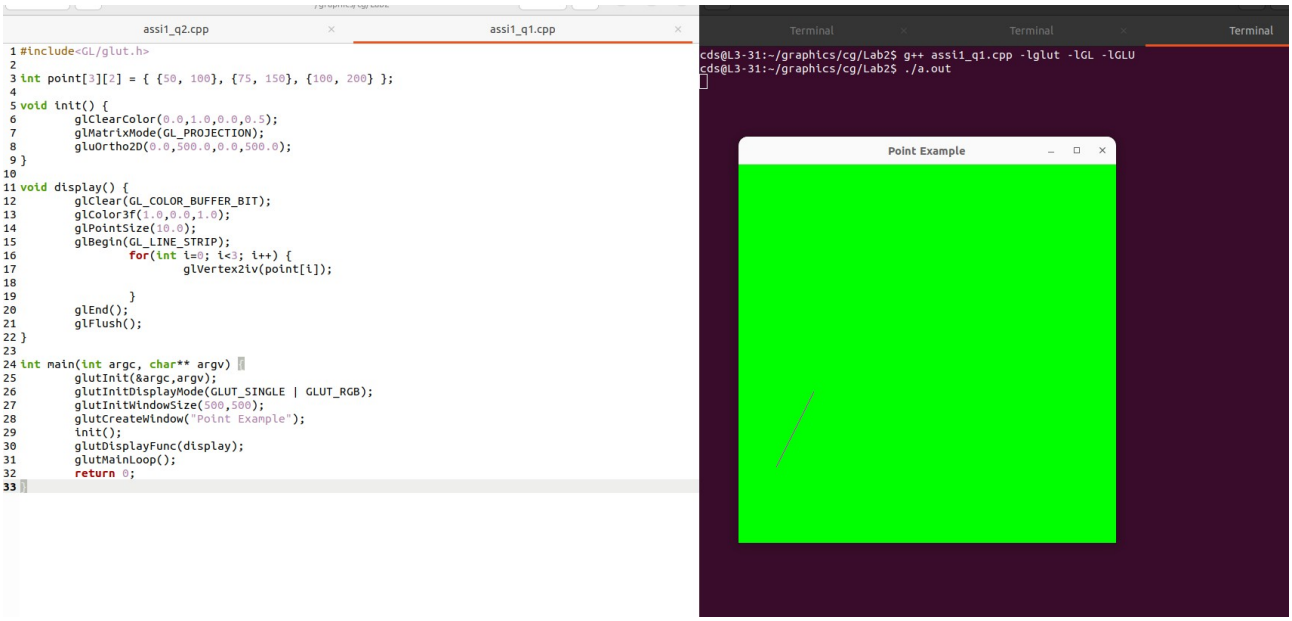
Computer Graphics Lab – 2

Assginment – 1

Q1.

CODE:

```
#include<GL/glut.h>

int point[3][2] = { {50, 100}, {75, 150}, {100, 200} };

void init() {
	glClearColor(0.0,1.0,0.0,0.5);
	glMatrixMode(GL_PROJECTION);
	gluOrtho2D(0.0,500.0,0.0,500.0);
}

void display() {
	glClear(GL_COLOR_BUFFER_BIT);
	glColor3f(1.0,0.0,1.0);
	glPointSize(10.0);
	glBegin(GL_LINE_STRIP);
		for(int i=0; i<3; i++) {
			glVertex2iv(point[i]);

		}
	glEnd();
	glFlush();
}

int main(int argc, char** argv) {
	glutInit(&argc,argv);
	glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
	glutInitWindowSize(500,500);
	glutCreateWindow("Point Example");
	init();
	glutDisplayFunc(display);
	glutMainLoop();
	return 0;
}
```



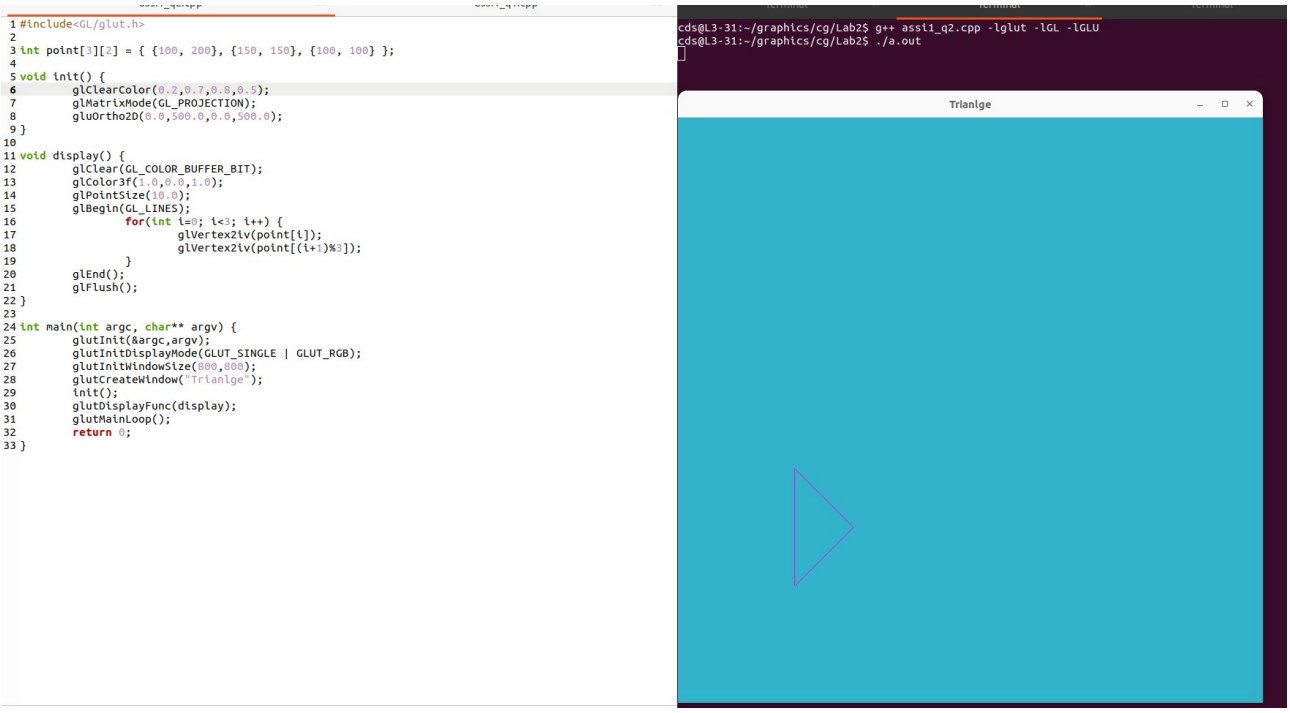
Q2 .

CODE:

```
#include<GL/glut.h>

int point[3][2] = { {100, 200}, {150, 150}, {100, 100} };

void init() {
	glClearColor(0.2,0.7,0.8,0.5);
	glMatrixMode(GL_PROJECTION);
	gluOrtho2D(0.0,500.0,0.0,500.0);
}

void display() {
	glClear(GL_COLOR_BUFFER_BIT);
	glColor3f(1.0,0.0,1.0);
	glPointSize(10.0);
	glBegin(GL_LINES);
		for(int i=0; i<3; i++) {
			glVertex2iv(point[i]);
			glVertex2iv(point[(i+1)%3]);
		}
	glEnd();
	glFlush();
}

int main(int argc, char** argv) {
	glutInit(&argc,argv);
	glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
	glutInitWindowSize(800,800);
	glutCreateWindow("Trianlge");
```

```
        init();
        glutDisplayFunc(display);
        glutMainLoop();
        return 0;
}
```

OUTPUT:

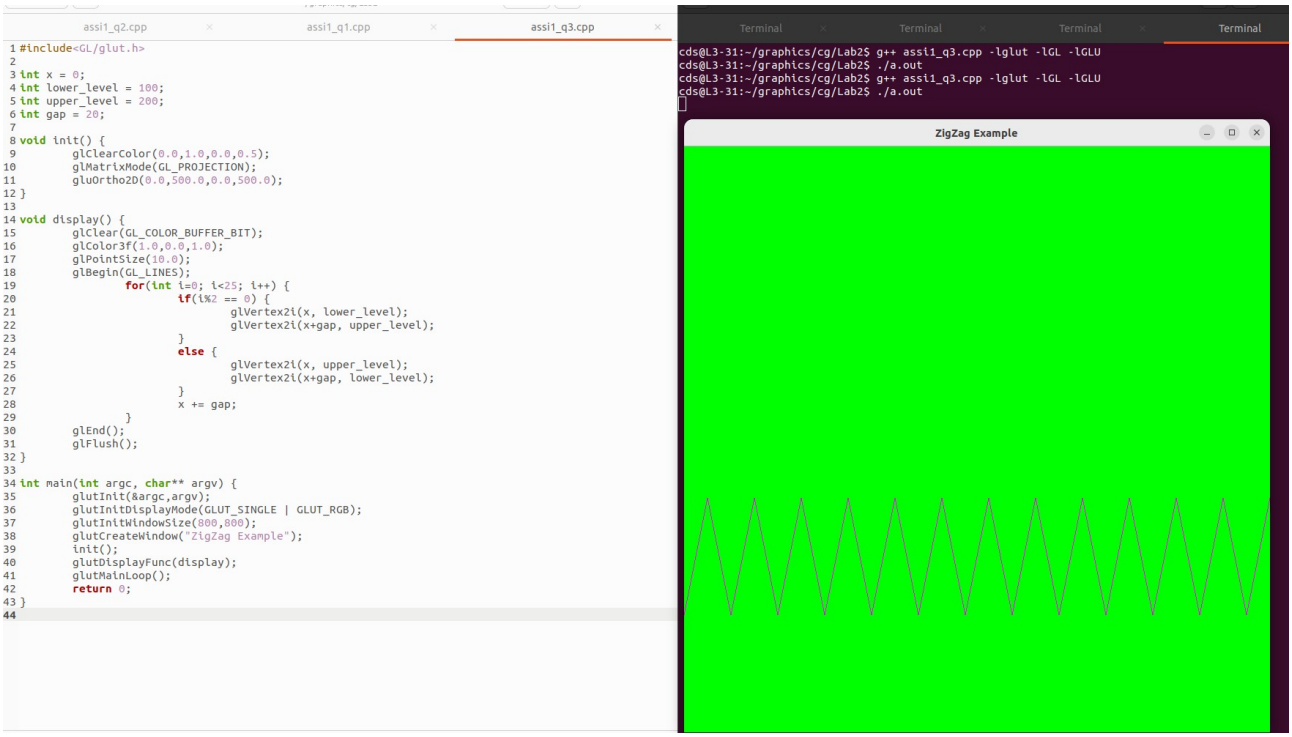


Q3.
CODE:

```
#include<GL/glut.h>

int x = 0;
int lower_level = 100;
int upper_level = 200;
int gap = 20;

void init() {
        glClearColor(0.0,1.0,0.0,0.5);
        glMatrixMode(GL_PROJECTION);
        gluOrtho2D(0.0,500.0,0.0,500.0);
}

void display() {
```

```
        glClear(GL_COLOR_BUFFER_BIT);
        glColor3f(1.0,0.0,1.0);
        glPointSize(10.0);
        glBegin(GL_LINES);
                for(int i=0; i<25; i++) {
                        if(i%2 == 0) {
                                glVertex2i(x, lower_level);
                                glVertex2i(x+gap, upper_level);
                        }
                        else {
                                glVertex2i(x, upper_level);
                                glVertex2i(x+gap, lower_level);
                        }
                        x += gap;
                }
        glEnd();
        glFlush();
}

int main(int argc, char** argv) {
        glutInit(&argc,argv);
        glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
        glutInitWindowSize(800,800);
        glutCreateWindow("ZigZag Example");
        init();
        glutDisplayFunc(display);
        glutMainLoop();
        return 0;
}
```

OUTPUT:

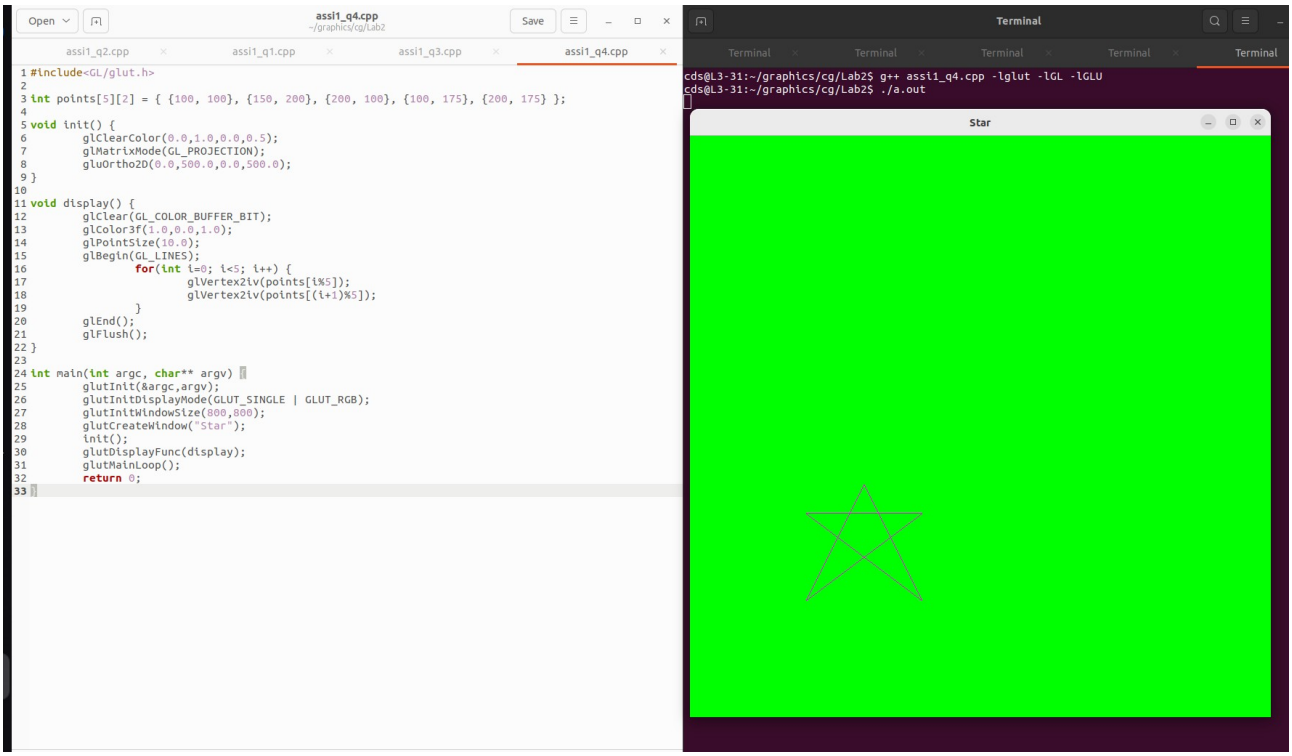


Q4.

CODE:

```
#include<GL/glut.h>

int points[5][2] = { {100, 100}, {150, 200}, {200, 100}, {100, 175}, {200, 175} };

void init() {
	glClearColor(0.0,1.0,0.0,0.5);
	glMatrixMode(GL_PROJECTION);
	gluOrtho2D(0.0,500.0,0.0,500.0);
}

void display() {
	glClear(GL_COLOR_BUFFER_BIT);
	glColor3f(1.0,0.0,1.0);
	glPointSize(10.0);
	glBegin(GL_LINES);
		for(int i=0; i<5; i++) {
			glVertex2iv(points[i%5]);
			glVertex2iv(points[(i+1)%5]);
		}
	glEnd();
	glFlush();
}

int main(int argc, char** argv) {
	glutInit(&argc,argv);
	glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
	glutInitWindowSize(800,800);
	glutCreateWindow("Star");
	init();
	glutDisplayFunc(display);
	glutMainLoop();
	return 0;
}
```

OUTPUT:



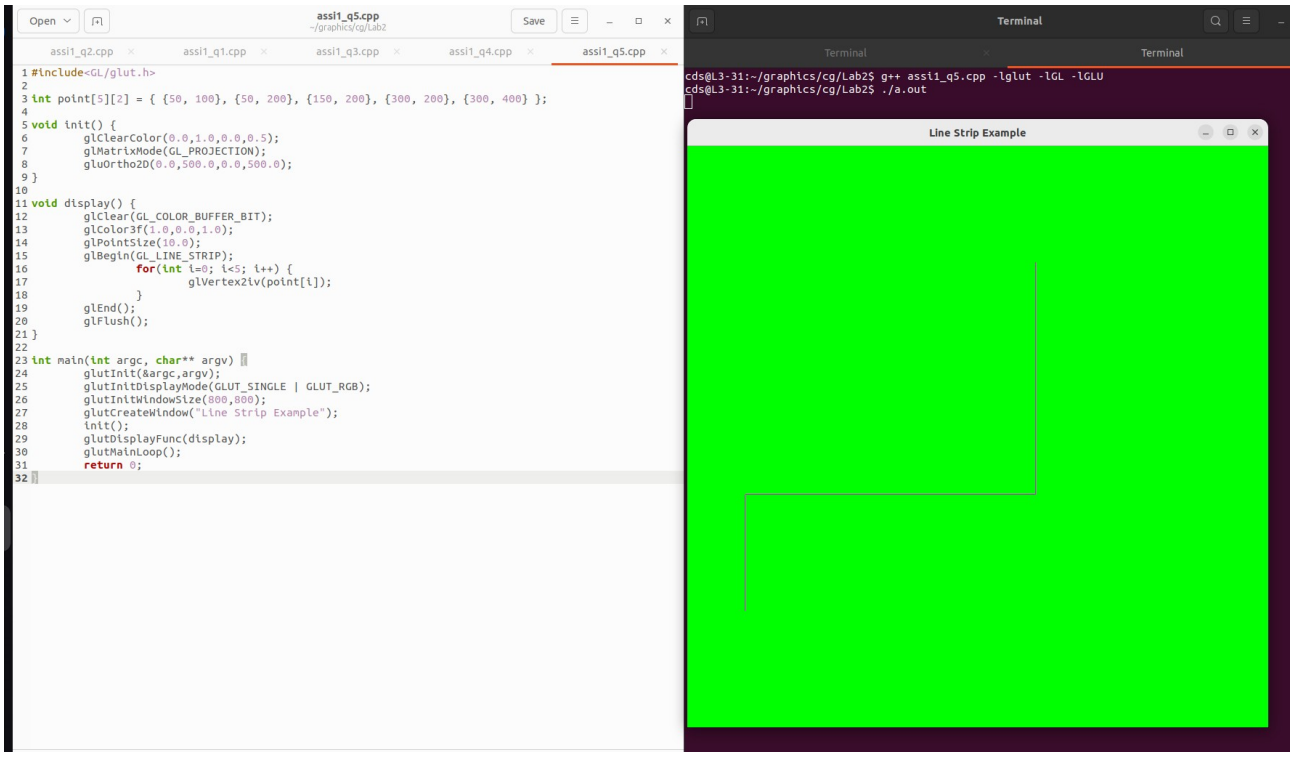
Q5.

CODE:

```
#include<GL/glut.h>

int point[5][2] = { {50, 100}, {50, 200}, {150, 200}, {300, 200}, {300, 400} };

void init() {
	glClearColor(0.0,1.0,0.0,0.5);
	glMatrixMode(GL_PROJECTION);
	gluOrtho2D(0.0,500.0,0.0,500.0);
}

void display() {
	glClear(GL_COLOR_BUFFER_BIT);
	glColor3f(1.0,0.0,1.0);
	glPointSize(10.0);
	glBegin(GL_LINE_STRIP);
		for(int i=0; i<5; i++) {
			glVertex2iv(point[i]);
		}
	glEnd();
	glFlush();
}

int main(int argc, char** argv) {
	glutInit(&argc,argv);
	glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
	glutInitWindowSize(800,800);
```

```
        glutCreateWindow("Line Strip Example");
        init();
        glutDisplayFunc(display);
        glutMainLoop();
        return 0;
}
```

OUTPUT:



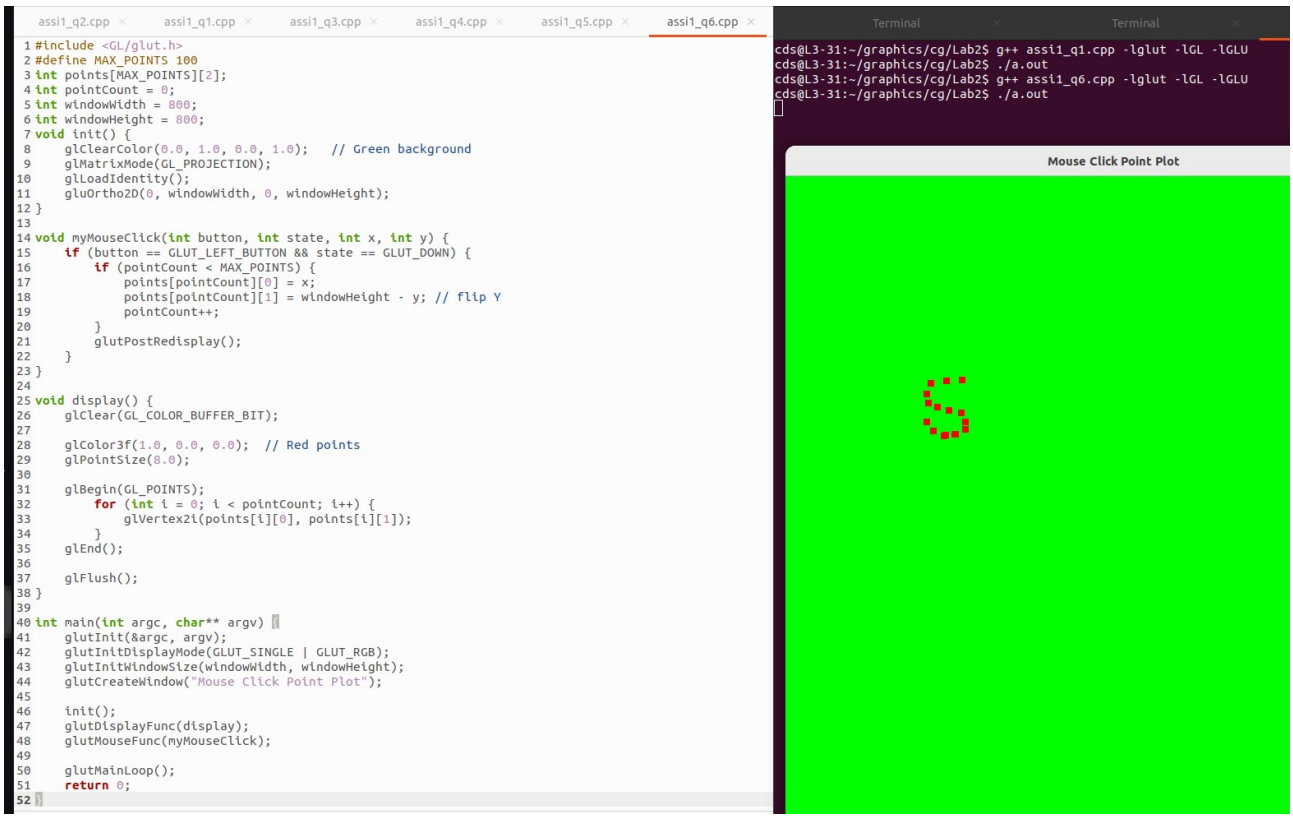
Q6.

CODE:

```
#include <GL/glut.h>
#define MAX_POINTS 100
int points[MAX_POINTS][2];
int pointCount = 0;
int windowWidth = 800;
int windowHeight = 800;
void init() {
    glClearColor(0.0, 1.0, 0.0, 1.0);   // Green background
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, windowWidth, 0, windowHeight);
```

```
}

void myMouseClick(int button, int state, int x, int y) {
    if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
        if (pointCount < MAX_POINTS) {
            points[pointCount][0] = x;
            points[pointCount][1] = windowHeight - y; // flip Y
            pointCount++;
        }
        glutPostRedisplay();
    }
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1.0, 0.0, 0.0);  // Red points
    glPointSize(8.0);

    glBegin(GL_POINTS);
        for (int i = 0; i < pointCount; i++) {
            glVertex2i(points[i][0], points[i][1]);
        }
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(windowWidth, windowHeight);
    glutCreateWindow("Mouse Click Point Plot");

    init();
    glutDisplayFunc(display);
    glutMouseFunc(myMouseClick);

    glutMainLoop();
    return 0;
}
```

OUTPUT:

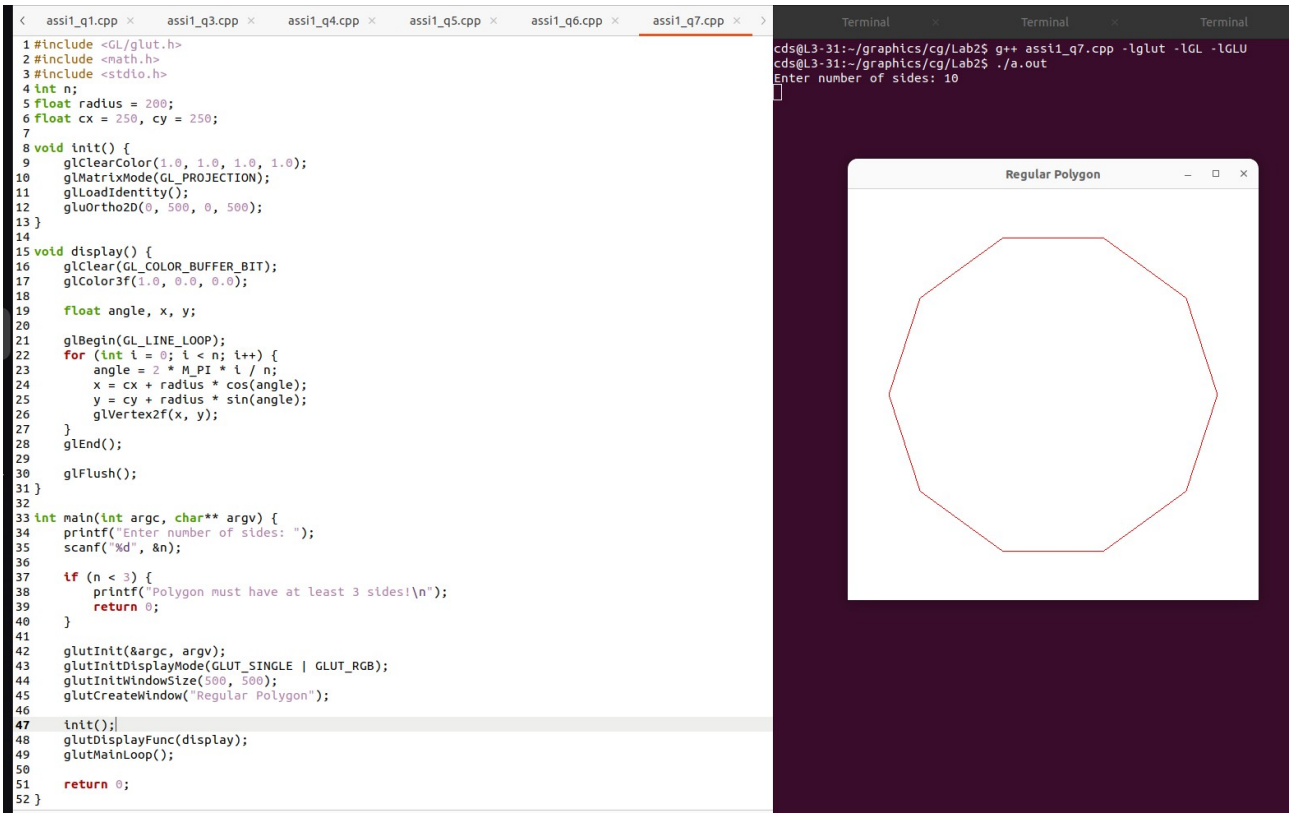


Q7.
CODE:

```
#include <GL/glut.h>
#include <math.h>
#include <stdio.h>
int n;
float radius = 200;
float cx = 250, cy = 250;

void init() {
   glClearColor(1.0, 1.0, 1.0, 1.0);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(0, 500, 0, 500);
}

void display() {
   glClear(GL_COLOR_BUFFER_BIT);
   glColor3f(1.0, 0.0, 0.0);

   float angle, x, y;

   glBegin(GL_LINE_LOOP);
   for (int i = 0; i < n; i++) {
      angle = 2 * M_PI * i / n;
      x = cx + radius * cos(angle);
```

```
        y = cy + radius * sin(angle);
        glVertex2f(x, y);
    }
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    printf("Enter number of sides: ");
    scanf("%d", &n);

    if (n < 3) {
        printf("Polygon must have at least 3 sides!\n");
        return 0;
    }

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("Regular Polygon");

    init();
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}
```

OUTPUT:



Q8.

CODE:

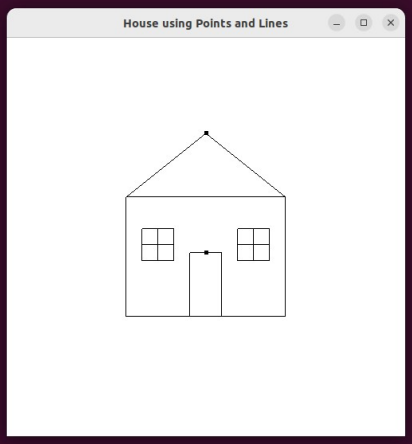
```
#include <GL/glut.h>
void init() {
   glClearColor(1.0, 1.0, 1.0, 1.0);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(0, 500, 0, 500);
}
void display() {
   glClear(GL_COLOR_BUFFER_BIT);
   glColor3f(0.0, 0.0, 0.0);

   /* ---- House Body ---- */
   glBegin(GL_LINE_LOOP);
      glVertex2i(150, 150);
      glVertex2i(350, 150);
      glVertex2i(350, 300);
      glVertex2i(150, 300);
   glEnd();

   /* ---- Roof ---- */
   glBegin(GL_LINES);
      glVertex2i(150, 300);
      glVertex2i(250, 380);

      glVertex2i(250, 380);
      glVertex2i(350, 300);
   glEnd();

   /* ---- Door ---- */
   glBegin(GL_LINE_LOOP);
      glVertex2i(230, 150);
      glVertex2i(270, 150);
      glVertex2i(270, 230);
      glVertex2i(230, 230);
   glEnd();

   /* ---- Left Window ---- */
   glBegin(GL_LINE_LOOP);
      glVertex2i(170, 220);
      glVertex2i(210, 220);
      glVertex2i(210, 260);
      glVertex2i(170, 260);
   glEnd();

   /* ---- Right Window ---- */
   glBegin(GL_LINE_LOOP);
      glVertex2i(290, 220);
```

```
        glVertex2i(330, 220);
        glVertex2i(330, 260);
        glVertex2i(290, 260);
    glEnd();

    /* ---- Window Cross Lines ---- */
    glBegin(GL_LINES);
        // Left window cross
        glVertex2i(190, 220);
        glVertex2i(190, 260);
        glVertex2i(170, 240);
        glVertex2i(210, 240);

        // Right window cross
        glVertex2i(310, 220);
        glVertex2i(310, 260);
        glVertex2i(290, 240);
        glVertex2i(330, 240);
    glEnd();

    /* ---- Decorative Points ---- */
    glPointSize(5.0);
    glBegin(GL_POINTS);
        glVertex2i(250, 380); // Roof top
        glVertex2i(250, 230); // Door top
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("House using Points and Lines");

    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:

assi1_q3.cpp | assi1_q4.cpp | assi1_q5.cpp | assi1_q6.cpp | assi1_q7.cpp | assi1_q8.cpp

```cpp
        glVertex2i(170, 220);
        glVertex2i(210, 220);
        glVertex2i(210, 260);
        glVertex2i(170, 260);
    glEnd();

    /* ---- Right Window ---- */
    glBegin(GL_LINE_LOOP);
        glVertex2i(290, 220);
        glVertex2i(330, 220);
        glVertex2i(330, 260);
        glVertex2i(290, 260);
    glEnd();

    /* ---- Window Cross Lines ---- */
    glBegin(GL_LINES);
        // Left window cross
        glVertex2i(190, 220);
        glVertex2i(190, 260);
        glVertex2i(170, 240);
        glVertex2i(210, 240);

        // Right window cross
        glVertex2i(310, 220);
        glVertex2i(310, 260);
        glVertex2i(290, 240);
        glVertex2i(330, 240);
    glEnd();

    /* ---- Decorative Points ---- */
    glPointSize(5.0);
    glBegin(GL_POINTS);
        glVertex2i(250, 380); // Roof top
        glVertex2i(250, 230); // Door top
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("House using Points and Lines");

    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

Terminal | Terminal | Terminal | Terminal

```
cds@L3-31:~/graphics/cg/Lab2$ g++ assi1_q8.cpp -lglut -lGL -lGLU
cds@L3-31:~/graphics/cg/Lab2$ ./a.out
```